

What This Project Is — and What Was Built

A plain-language companion to the formal report (`REPORT.md`). Read this one first — it explains the whole project as a story; `REPORT.md` gives the submission-ready depth, benchmarks and references.

1. The one-paragraph version

Three warehouse rovers are chained — by software — to one heavy pallet. No single rover can move it; they must **pull together**, stay **in formation**, and **not crash** into shelves, each other, or a robot that patrols the aisle. Nobody sits in a control room telling them where to go: each rover decides for itself using only what it can "see" around itself. I taught them to do this with **multi-agent deep reinforcement learning** (two algorithms, MAPPO and QMIX, implemented from scratch), compared them against a hand-engineered classical controller, and packaged everything as a real-time visual demo, formal benchmarks, figures, and a full report. The best learned policy (QMIX) delivers the payload **73% of the time** through the full obstacle course; the safest one (MAPPO) completes deliveries with **zero collisions**.

2. The problem, in everyday terms

Your proposal slide 2 gives the industry context: warehouses and farms are replacing one big machine with fleets of small cooperating robots. The core hard task is **cooperative transport** — moving something too heavy or bulky for one unit. Three things make it genuinely difficult at the same time:

1. **Coupled physics.** The rovers are tied to one shared body. If one pulls harder or turns, the payload rotates and drags everyone else with it.
2. **Decentralization.** Each rover sees only its own surroundings (a small LiDAR-like sensor, the payload, its teammates). There is no boss robot.
3. **Collision avoidance.** Cluttered arena, and one obstacle that *moves*.

Classical path planners solve this by computing everyone's path centrally — but they scale badly and fail when one unit dies or the world changes. The MARL bet: let the robots *learn* decentralized rules that add up to team intelligence.

3. What was actually built

`src/env.py` — **the world**. A 12×12 m arena with three differential-drive rovers, a 2×2 m rigid payload, five static obstacles and one patrolling one. The scientific heart is the **compliant shared rig**: each rover connects to its attachment point by a spring–damper, so forces are physical — yank and the payload swings, teammates get dragged, formation matters. Every rover gets a 26-number local observation (9 LiDAR beams + relative goal/payload/teammate vectors); a separate 27-number *global state* exists only for the learning algorithms' centralized components (this split is the CTDE paradigm).

`src/mappo.py` — **MAPPO**. Multi-agent PPO: one shared policy network (each rover runs the same brain on its own view), one centralized critic that sees the global state during training only. Beta-distribution policy for smooth continuous wheel commands, GAE advantages, clipped updates.

`src/qmix.py` — **QMIX**. Value-based: each rover has a Q-network scoring its own actions; a "mixing network" with enforced positive weights combines them into one team value that the team jointly maximizes. Discrete action grid (3×3 wheel levels), replay buffer, double-Q targets, epsilon-greedy exploration.

`src/baseline.py` — **the classical rival**. An artificial-potential-field controller: rovers aim for stations on the front side of the payload, steer around obstacles with tangential escape, and back off when stuck. It reads exact ground truth — making it a strong but *privileged* upper bound.

`src/train.py` — **the training engine**. Runs either algorithm with **curriculum learning** (stage 0: empty arena → stage 3: full obstacle course), periodic greedy evaluation, CSV logs, resumable checkpoints (`best.pt` keeps the

best policy ever seen, `final.pt` the latest). Training survives interruptions: re-run the same command and it continues.

`src/evaluate.py` — **the examiner**. Runs any policy over 30 fixed-seed episodes of the *full* task and reports success rate, collisions, path efficiency, rig stability, delivery speed — the metrics your slide 7 promised.

`src/demo.py` — **the showpiece**. Real-time PyGame visualization: rovers painted with their LiDAR rays, the payload and rig lines, the goal zone, live metrics, pause/reset/speed controls. Works for both trained models, the baseline, and an untrained random policy for contrast.

`src/plots.py`, `src/trajectories.py` — **the evidence**. Learning curves, benchmark bars, the curriculum-ablation chart, and trajectory maps showing *how* each policy actually drove the same scenario.

`REPORT.md` — **the submission document**. Formal structure: abstract, problem statement, architecture, algorithms, methodology, results with tables, discussion, future work, the four references from your proposal.

4. What the numbers say

30-episode formal benchmark on the hardest setting:

Policy	Success	Collisions/ep	Delivery time	Character
Random	0%	1.6	—	sanity floor
MAPPO	30%	0 on every success	427 steps	the careful one
QMIX	73%	42.9	182 steps	the fast pusher
APF baseline*	100%	2.8	81 steps	privileged classical

The most report-worthy findings:

- **Curriculum was decisive**. Without it MARL barely learned (12–25%); with it QMIX hit 100% on obstacle-free and stage-2 tasks and 73% on the full task. Same code, same compute — just staged difficulty.
- **Speed vs safety emerged naturally**. QMIX shoves through gaps (fast, many contacts); MAPPO glides (slow, zero collisions when it succeeds). Two different personalities from the same reward — a great discussion point.
- **The endgame is the frontier**. Most failures are near-misses: the payload arrives but the team doesn't hold it still long enough, especially when the moving obstacle disturbs the final approach. This is honestly documented as future work rather than hidden.
- **The baseline's 100% is not a fair fight — deliberately**. It sees everything; the learners see only 9 LiDAR beams. That gap quantifies the price of decentralization and motivates the whole research area.

5. Why this satisfies the proposal (slide by slide)

- **Slide 4 (architecture)**: `sim env` → CTDE training → deployment-style inference, exactly the pipeline the slide drew.
- **Slide 5 (environment)**: 2–4 rovers supported, shared-rig physics, constrained shared space, dynamic obstacle, domain randomization via seeds.
- **Slide 6 (MARL engine)**: local obs for actors, global state for centralized critic/mixer, cooperative reward engineering — all implemented.
- **Slide 7 (demo & metrics)**: real-time PyGame demo + the four promised metrics formally measured.
- **Slide 10 (roadmap)**: all four phases (`survey` → `environment` → `training + ablations` → `demo + metrics report`) are complete and evidenced in `runs/` and `plots/`.

6. How to demo it in 60 seconds (viva script)

1. `python src/demo.py --random` — chaos; shows the task is non-trivial.

2. `python src/demo.py --baseline` — classical controller delivers fast.
3. `python src/demo.py --ckpt runs/qmix_n3_s0/ckpt/final.pt` — the learned decentralized team doing it with local sensing only.
4. Open `plots/comparisonBars.png` and `plots/trajectories.png` — the numbers behind the story.

7. Honest limitations

2-D kinematic world (no wheel slip or payload tilt in 3-D), one dynamic obstacle, single training seed per algorithm, and MAPPO's endgame remains unconsolidated. Each is a named future-work item in `REPORT.md` §8 — and each is a fair, defensible answer if the panel asks.